

## 9 | PROBABILISTIC MODELING

The world is noisy and messy. You need to deal with the noise and uncertainty.

– Daphne Koller

MANY OF THE MODELS AND ALGORITHMS you have learned about thus far are relatively *disconnected*. There is an alternative view of machine learning that unites and generalizes much of what you have already learned. This is the **probabilistic modeling** framework, in which you will explicitly think of learning as a problem of **statistical inference**.

In this chapter, you will learn about two flavors of probabilistic models: generative and conditional. You will see that many of the approaches (both supervised and unsupervised) we have seen already can be cast as probabilistic models. Through this new view, you will be able to develop learning algorithms that have inductive biases closer to what you, as a designer, believe. Moreover, the two chapters that follow will make heavy use of the probabilistic modeling approach to open doors to other learning problems.

### Learning Objectives:

- Define the generative story for a naive Bayes classifier.
- Derive relative frequency as the solution to a constrained optimization problem.
- Compare and contrast generative, conditional and discriminative learning.
- Explain when generative models are likely to fail.
- Derive logistic loss with an  $\ell_2$  regularizer from a probabilistic perspective.

Dependencies:

### 9.1 Classification by Density Estimation

Recall from Chapter 2 that if we had access to the underlying probability distribution  $\mathcal{D}$ , then we could form a Bayes optimal classifier as:

$$f^{(\text{BO})}(\hat{x}) = \arg \max_{\hat{y} \in \mathcal{Y}} \mathcal{D}(\hat{x}, \hat{y}) \quad (9.1)$$

Unfortunately, no one gave you this distribution, but the optimality of this approach suggests that good way to build a classifier is to try to *estimate*  $\mathcal{D}$ . In other words, you try to learn a distribution  $\hat{\mathcal{D}}$ , which you hope to very similar to  $\mathcal{D}$ , and then use this distribution for classification. Just as in the preceding chapters, you can try to form your estimate of  $\mathcal{D}$  based on a finite training set.

The most direct way that you can attempt to construct such a probability distribution is to select a *family* of parametric distributions. For instance, a Gaussian (or Normal) distribution is parametric: its parameters are its mean and covariance. The job of learning is

**MATH REVIEW | RULES OF PROBABILITY**

A probability distribution  $p$  specifies the likelihood of an event  $e$ , where  $p(e) \in [0, 1]$ . It's often convenient to think of events as "configurations of the world", so  $p(e)$  says "how likely is it that the world is in configuration  $e$ ." Often world configurations are built up of smaller pieces, for instance you might say " $e$  = the configuration in which it is rainy, windy and cold." Formally, we might write this as " $e = \{\text{Weather} = \text{rainy}, \text{Wind} = \text{windy}, \text{Temperature} = \text{cold}\}$ ", where we've used a convention that **random variables** (like Temperature) are capitalized and their instantiations (like cold) are lower case. Considering this event, we want to evaluate  $p(\text{Weather} = \text{rainy}, \text{Wind} = \text{windy}, \text{Temperature} = \text{cold})$ , or more generally  $p(A = a, B = b, C = c)$  for some random variables  $A, B$  and  $C$ , and some instantiations of those random variables  $a, b$  and  $c$  respectively.

There are a few standard rules of probability that we will use regularly:

**sum-to-one:** if you sum over all possible configurations of the world,  $p$  sums to one:  $\sum_e p(E = e) = 1$ .

**marginalization:** you can sum out one random variable to remove it from the world:  $\sum_a p(A = a, B = b) = p(B = b)$ .

**chain rule:** if a world configuration consists of two or more random variables, you can evaluate the likelihood of the world one step at a time:  $p(A = a, B = b) = p(A = a)p(B = b | A = a)$ . Events are unordered, so you can also get  $p(A = a, B = b) = p(B = b)p(A = a | B = b)$ .

**Bayes rule:** combining the two chain rule equalities and dividing, we can relate a conditional probability in one direction with that in the other direction:  $p(A = a | B = b) = p(A = a)p(B = b | A = a) / p(B = b)$ .

Figure 9.1:

then to infer which parameters are "best" as far as the observed training data is concerned, as well as whatever inductive bias you bring. A key assumption that you will need to make is that the training data you have access to is drawn **independently** from  $\mathcal{D}$ . In particular, as you draw examples  $(x_1, y_1) \sim \mathcal{D}$  then  $(x_2, y_2) \sim \mathcal{D}$  and so on, the  $n$ th draw  $(x_n, y_n)$  is drawn from  $\mathcal{D}$  and *does not* otherwise depend on the previous  $n - 1$  samples. This assumption is usually false, but is also usually sufficiently close to being true to be useful. Together with the assumption that all the training data is drawn from the same distribution  $\mathcal{D}$  leads to the **i.i.d. assumption** or **independently and identically distributed** assumption. This is a key assumption in almost all of machine learning.

## 9.2 Statistical Estimation

Suppose you need to model a coin that is possibly biased (you can think of this as modeling the *label* in a binary classification problem), and that you observe data HHTH (where H means a flip came up heads

some unknown probability distribution of the coin

and T means it came up tails). You can assume that all the flips came from the same coin, and that each flip was independent (hence, the data was i.i.d.). Further, you may choose to believe that the coin has a fixed probability  $\beta$  of coming up heads (and hence  $1 - \beta$  of coming up tails). Thus, the parameter of your model is simply the scalar  $\beta$ .

The most basic computation you might perform is **maximum likelihood estimation**: namely, **select the parameter  $\beta$  that maximizes the probability of the data under that parameter**. In order to do so, you need to compute the probability of the data:

$$p_{\beta}(D) = p_{\beta}(\text{HHTH}) \quad \text{definition of } D \quad (9.2)$$

$$= p_{\beta}(\text{H})p_{\beta}(\text{H})p_{\beta}(\text{T})p_{\beta}(\text{H}) \quad \text{data is independent} \quad (9.3)$$

$$= \beta\beta(1 - \beta)\beta \quad (9.4)$$

$$= \beta^3(1 - \beta) \quad (9.5)$$

$$= \beta^3 - \beta^4 \quad (9.6)$$

each observation is independent and comes from the same distribution

MLE: approximate parameter (that describes a distribution) to maximize the likelihood of the observations coming from the distribution

?

Describe a case in which at least one of the assumptions we are making about the coin flip is false.

I want to find  $\beta$  that maximizes the probability of HHTH being observed

joint probability of the observations

This is the formula in terms of  $\beta$  for the probability of HHTH

Thus, if you want the parameter  $\beta$  that maximizes the probability of the data, you can take the derivative of  $\beta^3 - \beta^4$  with respect to  $\beta$ , set it equal to zero and solve for  $\beta$ :

$$\frac{\partial}{\partial \beta} [\beta^3 - \beta^4] = 3\beta^2 - 4\beta^3 = 0 \quad (9.7)$$

$$4\beta^3 = 3\beta^2 \quad (9.8)$$

$$\iff 4\beta = 3 \quad (9.9)$$

$$\iff \beta = \frac{3}{4} \quad (9.10)$$

Thus, the maximum likelihood  $\beta$  is 0.75, which is probably what you would have selected by intuition. You can solve this problem more generally as follows. If you have  $H$ -many heads and  $T$ -many tails, the probability of your data sequence is  $\beta^H(1 - \beta)^T$ . You can try to take the derivative of this with respect to  $\beta$  and follow the same recipe, but all of the products make things difficult. A more friendly solution is to work with the **log likelihood** or **log probability** instead. The log likelihood of this data sequence is  $H \log \beta + T \log(1 - \beta)$ . Differentiating with respect to  $\beta$ , you get  $H/\beta - T/(1 - \beta)$ . To solve, you obtain  $H/\beta = T/(1 - \beta)$  so  $H(1 - \beta) = T\beta$ . Thus  $H - H\beta = T\beta$  and so  $H = (H + T)\beta$ , finally yielding that  $\beta = H/(H + T)$  or, simply, the fraction of observed data that came up heads. In this case, the maximum likelihood estimate is nothing but the relative frequency of observing heads!

same as  
# of heads / total # of flips

Now, suppose that instead of flipping a coin, you're rolling a  $K$ -sided die (for instance, to pick the label for a multiclass classification problem). You might model this by saying that there are parameters  $\theta_1, \theta_2, \dots, \theta_K$  specifying, respectively, the probabilities that any given

?

How do you know that the solution of  $\beta = H/(H + T)$  is actually a maximum?

side comes up on a role. Since these are themselves probabilities, each  $\theta_k$  should be at least zero, and the sum of the  $\theta_k$ s should be one. Given a data set that consists of  $x_1$  rolls of 1,  $x_2$  rolls of 2 and so on, the probability of this data is  $\prod_k \theta_k^{x_k}$ , yielding a log probability of  $\sum_k x_k \log \theta_k$ . If you pick some particular parameter, say  $\theta_3$ , the derivative of this with respect to  $\theta_3$  is  $x_3/\theta_3$ , which you want to equate to zero. This leads to  $\dots \theta_3 \rightarrow \infty$ .

This is obviously “wrong.” From the mathematical formulation, it’s correct: in fact, setting all of the  $\theta_k$ s to  $\infty$  *does* maximize  $\prod_k \theta_k^{x_k}$  for any (non-negative)  $x_k$ s. The problem is that you need to constrain the  $\theta$ s to sum to one. In particular, you have a constraint that  $\sum_k \theta_k = 1$  that you forgot to enforce. A convenient way to enforce such constraints is through the technique of **Lagrange multipliers**. To make this problem consistent with standard minimization problems, it is convenient to minimize negative log probabilities, instead of maximizing log probabilities. Thus, the *constrained* optimization problem is:

$$\begin{aligned} \min_{\theta} \quad & -\sum_k x_k \log \theta_k \\ \text{subj. to} \quad & \sum_k \theta_k - 1 = 0 \end{aligned} \tag{9.11}$$

The Lagrange multiplier approach involves adding a new variable  $\lambda$  to the problem (called the **Lagrange variable**) corresponding to the constraint, and to use that to move the constraint into the objective. The result, in this case, is:

$$\max_{\lambda} \min_{\theta} \quad -\sum_k x_k \log \theta_k - \lambda \left( \sum_k \theta_k - 1 \right) \tag{9.12}$$

Turning a constrained optimization problem into its corresponding **Lagrangian** is straightforward. The mystical aspect is why it works. In this case, the idea is as follows. Think of  $\lambda$  as an adversary:  $\lambda$  is trying to maximize this function (you’re trying to minimize it). If you pick some parameters  $\theta$  that actually satisfy the constraint, then the green term in Eq (9.12) goes to zero, and therefore  $\lambda$  does not matter: the adversary cannot do anything. On the other hand, if the constraint is even *slightly* unsatisfied, then  $\lambda$  can tend toward  $+\infty$  or  $-\infty$  to blow up the objective. So, in order to have a non-infinite objective value, the optimizer *must* find values of  $\theta$  that satisfy the constraint.

If we solve the *inner* optimization of Eq (9.12) by differentiating with respect to  $\theta_1$ , we get  $x_1/\theta_1 = \lambda$ , yielding  $\theta_1 = x_1/\lambda$ . In general, the solution is  $\theta_k = x_k/\lambda$ . Remembering that the goal of  $\lambda$  is to enforce the sums-to-one constraint, we can set  $\lambda = \sum_k x_k$  and verify

that this is a solution. Thus, our optimal  $\theta_k = x_k / \sum_k x_k$ , which again completely corresponds to intuition.

### 9.3 Naive Bayes Models

Now, consider the binary classification problem. You are looking for a **parameterized probability distribution** that can describe the training data you have. To be concrete, your task might be to predict whether a movie review is positive or negative (label) based on what words (features) appear in that review. Thus, the probability for a *single* data point can be written as:

Binary classification:  
- probability distribution of labels (y) but with respect to x

Earlier: roll1 = H roll2 = H roll3 = T roll4 = H  
Maximized joint probability of p(HHTH)

$$p_{\theta}((y, \mathbf{x})) = p_{\theta}(y, x_1, x_2, \dots, x_D) \quad (9.13)$$

This is one observation of D variables and a label  
You want to maximize joint probability of p(y, x1, x2, ..., xD)

The challenge in working with a probability distribution like Eq (9.13) is that it's a distribution over a *lot* of variables. You can try to simplify it by applying the **chain rule** of probabilities:

$$p_{\theta}(x_1, x_2, \dots, x_D, y) = p_{\theta}(y) p_{\theta}(x_1 | y) p_{\theta}(x_2 | y, x_1) p_{\theta}(x_3 | y, x_1, x_2) \dots p_{\theta}(x_D | y, x_1, x_2, \dots, x_{D-1}) \quad (9.14)$$

$$= p_{\theta}(y) \prod_d p_{\theta}(x_d | y, x_1, \dots, x_{d-1}) \quad (9.15)$$

At this point, this equality is *exact* for any probability distribution. However, it might be difficult to craft a probability distribution for the 10000th feature, given the previous 9999. Even if you could, it might be difficult to accurately estimate it. At this point, you can make assumptions. A classic assumption, called the **naive Bayes assumption**, is that **the features are independent, conditioned on the label**.

assumption :  
conditional independence of feature values

In the movie review example, this is saying that *once you know that it's a positive review*, the probability that the word “excellent” appears is independent of whether “amazing” also appeared. (Note that this does *not* imply that these words are independent when you don't know the label—they most certainly are not.) Formally this assumption states that:

$$\textbf{Assumption: } p(x_d | y, x_{d'}) = p(x_d | y) \quad , \quad \forall d \neq d' \quad (9.16)$$

Under this assumption, you can simplify Eq (9.15) to:

$$p_{\theta}((y, \mathbf{x})) = p_{\theta}(y) \prod_d p_{\theta}(x_d | y) \quad \textbf{naive Bayes assumption} \quad (9.17)$$

At this point, you can start parameterizing  $p$ . Suppose, for now, that your labels are binary *and* your features are also binary. In this case, you could model the label as a biased coin, with probability of heads (e.g., positive review) given by  $\theta_0$ . Then, for each label, you

can imagine having one (biased) coin for each feature. So if there are  $D$ -many features, you'll have  $1 + 2D$  total coins: one for the label (call it  $\theta_0$ ) and one for each label/feature combination (call these  $\theta_{+1}$  and as  $\theta_{-1}$ ). In the movie review example, we might expect  $\theta_0 \approx 0.4$  (forty percent of movie reviews are positive) and also that  $\theta_{+1}$  might give high probability to words like “excellent” and “amazing” and “good” and  $\theta_{-1}$  might give high probability to words like “terrible” and “boring” and “hate”. You can rewrite the probability of a single example as follows, eventually leading to the log probability of the entire data set:

$$p_{\theta}(y, x) = p_{\theta}(y) \prod_d p_{\theta}(x_d | y) \quad \text{naive Bayes assumption} \quad (9.18)$$

$$= \left( \theta_0^{[y=+1]} (1 - \theta_0)^{[y=-1]} \right) \prod_d \theta_{(y),d}^{[x_d=1]} (1 - \theta_{(y),d})^{[x_d=0]} \quad \text{model assumptions} \quad (9.19)$$

# of pos labels / total labels or observations

Solving for  $\theta_0$  is identical to solving for the biased coin case from before: it is just the relative frequency of positive labels in your data (because  $\theta_0$  doesn't depend on  $x$  at all). For the other parameters, you can repeat the same exercise as before for each of the  $2D$  coins independently. This yields:

$$p(y = \text{pos}) \quad \hat{\theta}_0 = \frac{1}{N} \sum_n [y_n = +1] \quad \frac{\text{\# of pos examples}}{\text{total \# of examples}} \quad p(y = \text{neg}) = 1 - p(y = \text{pos}) \quad (9.20) \quad 1 \text{ param}$$

$$\hat{\theta}_{(+1),d} = \frac{\sum_n [y_n = +1 \wedge x_{n,d} = 1]}{\sum_n [y_n = +1]} \quad \frac{\text{\# of obs where } x_d = 1 \text{ (knowing } y = \text{pos)}}{\text{\# of pos examples}} \quad (9.21) \quad d \text{ params}$$

$$\hat{\theta}_{(-1),d} = \frac{\sum_n [y_n = -1 \wedge x_{n,d} = 1]}{\sum_n [y_n = -1]} \quad \frac{\text{\# of obs where } x_d = 1 \text{ (knowing } y = \text{neg)}}{\text{\# of neg examples}} \quad (9.22) \quad d \text{ params}$$

In the case that the features are *not* binary, you need to choose a different model for  $p(x_d | y)$ . The model we chose here is the **Bernoulli distribution**, which is effectively a distribution over independent coin flips. For other types of data, other distributions become more appropriate. The die example from before corresponds to a **discrete distribution**. If the data is continuous, you might choose to use a **Gaussian distribution** (aka **Normal distribution**). The choice of distribution is a form of **inductive bias** by which you can inject your knowledge of the problem into the learning algorithm.

## 9.4 Prediction

Consider the *predictions* made by the naive Bayes model with Bernoulli features in Eq (9.18). You can better understand this model by considering its decision boundary. In the case of probabilistic models,

### MATH REVIEW | COMMON PROBABILITY DISTRIBUTIONS

There are a few common probability distributions that we use in this book. The first is the Bernoulli distribution, which models binary outcomes (like coin flips). A Bernoulli distribution,  $\mathcal{Ber}(\theta)$  is parameterized by a single scalar value  $\theta \in [0, 1]$  that represents the probability of heads. The likelihood function is  $\mathcal{Ber}(x | \theta) = \theta^x(1 - \theta)^{1-x}$ . The generalization of the Bernoulli to more than two possible outcomes (like rolls of a die) is the Discrete distribution,  $\mathcal{Disc}(\theta)$ . If the die has  $K$  sides, then  $\theta \in \mathbb{R}^K$  with all entries non-negative and  $\sum_k \theta_k = 1$ .  $\theta_k$  is the probability that the die comes up on side  $k$ . The likelihood function is  $\mathcal{Disc}(x | \theta) = \prod_k \theta_k^{1[x=k]}$ . The Binomial distribution is just like the Bernoulli distribution but for multiple flips of the rather than a single flip; its likelihood is  $\binom{n}{k} \theta^k (1-\theta)^{n-k}$ , where  $n$  is the number of flips and  $k$  is the number of heads. The Multinomial distribution extends the Discrete distribution also to multiple rolls; its likelihood is  $\frac{n!}{\prod_k x_k!} \prod_k \theta_k^{x_k}$ , where  $n$  is the total number of rolls and  $x_k$  is the number of times the die came up on side  $k$  (so  $\sum_k x_k = n$ ). The preceding distributions are all discrete.

There are two common continuous distributions we need. The first is the Uniform distribution,  $\mathcal{Uni}(a, b)$  which is uniform over the closed range  $[a, b]$ . Its density function is  $\mathcal{Uni}(x | a, b) = \frac{1}{b-a} \mathbf{1}[x \in [a, b]]$ . Finally, the Gaussian distribution is parameterized by a mean  $\mu$  and variance  $\sigma^2$  and has density  $\mathcal{Nor}(x | \mu, \sigma^2) = (2\pi\sigma^2)^{-\frac{1}{2}} \exp\left[-\frac{1}{2\sigma^2}(x - \mu)^2\right]$ .

Figure 9.2:

the decision boundary is the set of inputs for which the likelihood of  $y = +1$  is precisely 50%. Or, in other words, the set of inputs  $x$  for which  $p(y = +1 | x) / p(y = -1 | x) = 1$ . In order to do this, the first thing to notice is that  $p(y | x) = p(y, x) / p(x)$ . In the ratio, the  $p(x)$  terms cancel, leaving  $p(y = +1, x) / p(y = -1, x)$ . Instead of computing this ratio, it is easier to compute the **log-likelihood ratio** (or LLR),  $\log p(y = +1, x) - \log p(y = -1, x)$ , computed below:

$$\begin{aligned} \text{LLR} &= \log \left[ \theta_0 \prod_d \theta_{(+1),d}^{[x_d=1]} (1 - \theta_{(+1),d})^{[x_d=0]} \right] \\ &\quad - \log \left[ (1 - \theta_0) \prod_d \theta_{(-1),d}^{[x_d=1]} (1 - \theta_{(-1),d})^{[x_d=0]} \right] \end{aligned} \tag{9.23}$$

model assumptions

$$\begin{aligned} &= \log \theta_0 - \log(1 - \theta_0) + \sum_d [x_d = 1] \left( \log \theta_{(+1),d} - \log \theta_{(-1),d} \right) \\ &\quad + \sum_d [x_d = 0] \left( \log(1 - \theta_{(+1),d}) - \log(1 - \theta_{(-1),d}) \right) \end{aligned} \tag{9.24}$$

take logs and rearrange

$$\begin{aligned} &= \sum_d x_d \log \frac{\theta_{(+1),d}}{\theta_{(-1),d}} + \sum_d (1 - x_d) \log \frac{1 - \theta_{(+1),d}}{1 - \theta_{(-1),d}} + \log \frac{\theta_0}{1 - \theta_0} \end{aligned} \tag{9.25}$$

simplify log terms

$$= \sum_d x_d \left[ \log \frac{\theta_{(+1),d}}{\theta_{(-1),d}} - \log \frac{1 - \theta_{(+1),d}}{1 - \theta_{(-1),d}} \right] + \sum_d \log \frac{1 - \theta_{(+1),d}}{1 - \theta_{(-1),d}} + \log \frac{\theta_0}{1 - \theta_0} \quad \text{group } x\text{-terms} \quad (9.26)$$

$$= \mathbf{x} \cdot \mathbf{w} + b \quad (9.27)$$

$$w_d = \log \frac{\theta_{(+1),d}(1 - \theta_{(-1),d})}{\theta_{(-1),d}(1 - \theta_{(+1),d})} \quad , \quad b = \sum_d \log \frac{1 - \theta_{(+1),d}}{1 - \theta_{(-1),d}} + \log \frac{\theta_0}{1 - \theta_0} \quad (9.28)$$

The result of the algebra is that the naive Bayes model has precisely the form of a linear model! Thus, like perceptron and many of the other models you've previous studied, the decision boundary is linear.

## 9.5 Generative Stories

A useful way to develop probabilistic models is to tell a **generative story**. This is a *fictional* story that explains how you believe your training data came into existence. To make things interesting, consider a multiclass classification problem, with continuous features modeled by independent Gaussians. Since the label can take values  $1 \dots K$ , you can use a discrete distribution (die roll) to model it (as opposed to the Bernoulli distribution from before):

1. **For each example**  $n = 1 \dots N$ :
  - (a) **Choose a label**  $y_n \sim \text{Disc}(\boldsymbol{\theta})$
  - (b) **For each feature**  $d = 1 \dots D$ :
    - i. **Choose feature value**  $x_{n,d} \sim \text{Nor}(\mu_{y_n,d}, \sigma_{y_n,d}^2)$

This generative story can be directly translated into a likelihood function by replacing the “for each”s with products:

$$p(D) = \overbrace{\prod_n \underbrace{\theta_{y_n}}_{\text{choose label}} \prod_d \underbrace{\frac{1}{\sqrt{2\pi\sigma_{y_n,d}^2}} \exp \left[ -\frac{1}{2\sigma_{y_n,d}^2} (x_{n,d} - \mu_{y_n,d})^2 \right]}_{\substack{\text{choose feature value} \\ \text{for each feature}}}}^{\text{for each example}} \quad (9.29)$$

You can take logs to arrive at the log-likelihood:

$$\log p(D) = \sum_n \left[ \log \theta_{y_n} + \sum_d -\frac{1}{2} \log(\sigma_{y_n,d}^2) - \frac{1}{2\sigma_{y_n,d}^2} (x_{n,d} - \mu_{y_n,d})^2 \right] + \text{const} \quad (9.30)$$

To optimize for  $\theta$ , you need to add a “sums to one” constraint as before. This leads to the previous solution where the  $\theta_k$ s are proportional to the number of examples with label  $k$ . In the case of the  $\mu$ s you can take a derivative with respect to, say  $\mu_{k,i}$  and obtain:

$$\frac{\partial \log p(D)}{\partial \mu_{k,i}} = \frac{\partial}{\partial \mu_{k,i}} - \sum_n \sum_d \frac{1}{2\sigma_{y_n,d}^2} (x_{n,d} - \mu_{y_n,d})^2 \quad \text{ignore irrelevant terms} \quad (9.31)$$

$$= \frac{\partial}{\partial \mu_{k,i}} - \sum_{n:y_n=k} \frac{1}{2\sigma_{k,d}^2} (x_{n,i} - \mu_{k,i})^2 \quad \text{ignore irrelevant terms} \quad (9.32)$$

$$= \sum_{n:y_n=k} \frac{1}{\sigma_{k,d}^2} (x_{n,i} - \mu_{k,i}) \quad \text{take derivative} \quad (9.33)$$

Setting this equal to zero and solving yields:

$$\mu_{k,i} = \frac{\sum_{n:y_n=k} x_{n,i}}{\sum_{n:y_n=k} 1} \quad (9.34)$$

Namely, the sample mean of the  $i$ th feature of the data points that fall in class  $k$ . A similar analysis for  $\sigma_{k,i}^2$  yields:

$$\frac{\partial \log p(D)}{\partial \sigma_{k,i}^2} = \frac{\partial}{\partial \sigma_{k,i}^2} - \sum_{y:y_n=k} \left[ \frac{1}{2} \log(\sigma_{k,i}^2) + \frac{1}{2\sigma_{k,i}^2} (x_{n,i} - \mu_{k,i})^2 \right] \quad \text{ignore irrelevant terms} \quad (9.35)$$

$$= - \sum_{y:y_n=k} \left[ \frac{1}{2\sigma_{k,i}^2} - \frac{1}{2(\sigma_{k,i}^2)^2} (x_{n,i} - \mu_{k,i})^2 \right] \quad \text{take derivative} \quad (9.36)$$

$$= \frac{1}{2\sigma_{k,i}^4} \sum_{y:y_n=k} \left[ (x_{n,i} - \mu_{k,i})^2 - \sigma_{k,i}^2 \right] \quad \text{simplify} \quad (9.37)$$

You can now set this equal to zero and solve, yielding:

$$\sigma_{k,i}^2 = \frac{\sum_{n:y_n=k} (x_{n,i} - \mu_{k,i})^2}{\sum_{n:y_n=k} 1} \quad (9.38)$$

Which is just the sample variance of feature  $i$  for class  $k$ .

## 9.6 Conditional Models

In the foregoing examples, the task was formulated as attempting to model the **joint** distribution of  $(x, y)$  pairs. This may seem wasteful: at prediction time, all you care about is  $p(y | x)$ , so why not model it directly?

?

What would the estimate be if you decided that, for a given class  $k$ , all features had equal variance? What if you assumed feature  $i$  had equal variance for each class? Under what circumstances might it be a good idea to make such assumptions?

Starting with the case of regression is actually somewhat simpler than starting with classification in this case. Suppose you “believe” that the relationship between the real value  $y$  and the vector  $x$  should be linear. That is, you expect that  $y = w \cdot x + b$  should hold for some parameters  $(w, b)$ . Of course, the data that you get does not exactly obey this: that’s fine, you can think of deviations from  $y = w \cdot x + b$  as *noise*. To form a probabilistic model, you must assume some distribution over noise; a convenient choice is zero-mean Gaussian noise. This leads to the following generative story:

1. For each example  $n = 1 \dots N$ :

- (a) Compute  $t_n = w \cdot x_n + b$
- (b) Choose noise  $e_n \sim \text{Nor}(0, \sigma^2)$
- (c) Return  $y_n = t_n + e_n$

In this story, the variable  $t_n$  stands for “target.” It is the noiseless variable that you do not get to observe. Similarly  $e_n$  is the error (noise) on example  $n$ . The value that you actually get to observe is  $y_n = t_n + e_n$ . See Figure 9.3.

A basic property of the Gaussian distribution is additivity. Namely, that if  $a \sim \text{Nor}(\mu, \sigma^2)$  and  $b = a + c$ , then  $b \sim \text{Nor}(\mu + c, \sigma^2)$ . Given this, from the generative story above, you can derive a shorter generative story:

1. For each example  $n = 1 \dots N$ :

- (a) Choose  $y_n \sim \text{Nor}(w \cdot x_n + b, \sigma^2)$

Reading off the log likelihood of a dataset from this generative story, you obtain:

$$\log p(D) = \sum_n \left[ -\frac{1}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} (w \cdot x_n + b - y_n)^2 \right] \quad \text{model assumptions} \quad (9.39)$$

$$= -\frac{1}{2\sigma^2} \sum_n (w \cdot x_n + b - y_n)^2 + \text{const} \quad \text{remove constants} \quad (9.40)$$

This is precisely the linear regression model you encountered in Section 8.6! To minimizing the *negative* log probability, you need only solve for the regression coefficients  $w, b$  as before.

In the case of binary classification, using a Gaussian noise model does not make sense. Switching to a Bernoulli model, which describes binary outcomes, makes more sense. The only remaining difficulty is that the parameter of a Bernoulli is a value between zero and one (the probability of “heads”) so your model must produce

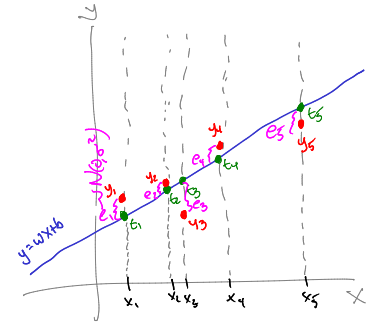


Figure 9.3: pictorial view of targets versus labels

such values. A classic approach is to produce a real-valued target, as before, and then transform this target into a value between zero and one, so that  $-\infty$  maps to 0 and  $+\infty$  maps to 1. A function that does this is the logistic function<sup>1</sup>, defined below and plotted in Figure 9.4:

$$\text{Logistic function: } \sigma(z) = \frac{1}{1 + \exp[-z]} = \frac{\exp z}{1 + \exp z} \quad (9.41)$$

The logistic function has several nice properties that you can verify for yourself:  $\sigma(-z) = 1 - \sigma(z)$  and  $\partial\sigma/\partial z = z\sigma^2(z)$ .

Using the logistic function, you can write down a generative story for binary classification:

1. For each example  $n = 1 \dots N$ :
  - (a) Compute  $t_n = \sigma(\mathbf{w} \cdot \mathbf{x}_n + b)$
  - (b) Compute  $z_n \sim \text{Ber}(t_n)$
  - (c) Return  $y_n = 2z_n - 1$  (to make it  $\pm 1$ )

The log-likelihood for this model is:

$$\begin{aligned} \log p(D) &= \sum_n \left[ [y_n = +1] \log \sigma(\mathbf{w} \cdot \mathbf{x}_n + b) \right. \\ &\quad \left. + [y_n = -1] \log \sigma(-\mathbf{w} \cdot \mathbf{x}_n + b) \right] \end{aligned} \quad \begin{array}{l} \text{model and properties of } \sigma \\ (9.42) \end{array}$$

$$= \sum_n \log \sigma(y_n (\mathbf{w} \cdot \mathbf{x}_n + b)) \quad \begin{array}{l} \text{join terms} \\ (9.43) \end{array}$$

$$= - \sum_n \log [1 + \exp(-y_n (\mathbf{w} \cdot \mathbf{x}_n + b))] \quad \begin{array}{l} \text{definition of } \sigma \\ (9.44) \end{array}$$

$$= - \sum_n \ell^{(\log)}(y_n, \mathbf{w} \cdot \mathbf{x}_n + b) \quad \begin{array}{l} \text{definition of } \ell^{(\log)} \\ (9.45) \end{array}$$

As you can see, the log-likelihood is *precisely* the negative of (a scaled version of) the logistic loss from Chapter 8. This model is the **logistic regression** model, and this is where logisitic loss originally derived from.

TODO: conditional versus joint

<sup>1</sup> Also called the **sigmoid function** because of its “S”-shape.

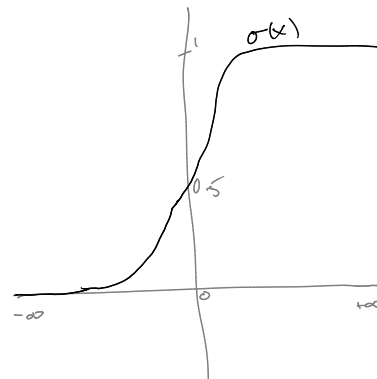


Figure 9.4: sketch of logistic function

## 9.7 Regularization via Priors

In the foregoing discussion, parameters of the model were selected according to the maximum likelihood criteria: find the parameters  $\theta$  that maximize  $p_\theta(D)$ . The trouble with this approach is easy to

see even in a simple coin flipping example. If you flip a coin twice and it comes up heads both times, the maximum likelihood estimate for the bias of the coin is 100%: it will *always* come up heads. This is true even if you had only flipped it once! Of course if you had flipped it one million times and it had come up heads every time, *then* you might find this to be a reasonable solution.

This is clearly undesirable behavior, especially since data is expensive in a machine learning setting. One solution (there are others!) is to seek parameters that balance a tradeoff between the likelihood of the data and some prior belief you have about what values of those parameters are likely. Taking the case of the logistic regression, you might a priori believe that small values of  $w$  are more likely than large values, and choose to represent this as a Gaussian prior on each component of  $w$ .

The **maximum a posteriori** principle is a method for incorporating both data and prior beliefs to obtain a more balanced parameter estimate. In abstract terms, consider a probabilistic model over data  $D$  that is parameterized by parameters  $\theta$ . If you think of the parameters as just another random variable, then you can write this model as  $p(D | \theta)$ , and maximum likelihood amounts to choosing  $\theta$  to maximize  $p(D | \theta)$ . However, you might instead wish to maximize the probability of the *parameters*, given the data. Namely, maximize  $p(\theta | D)$ . This term is known as the **posterior** distribution on  $\theta$ , and can be computed by Bayes' rule:

$$\underbrace{p(\theta | D)}_{\text{posterior}} = \frac{\overbrace{p(\theta)}^{\text{prior}} \overbrace{p(D | \theta)}^{\text{likelihood}}}{\underbrace{p(D)}_{\text{evidence}}}, \text{ where } p(D) = \int d\theta p(\theta) p(D | \theta) \quad (9.46)$$

This reads: the **posterior** is equal to the **prior** times the **likelihood** divided by the **evidence**.<sup>2</sup> The evidence is a scary-looking term (it has an integral!) but note that from the perspective of seeking parameters  $\theta$  than maximize the posterior, the evidence is just a constant (it does not depend on  $\theta$ ) and therefore can be ignored.

<sup>2</sup> The evidence is sometimes called the **marginal likelihood**.

Returning to the logistic regression example with Gaussian priors on the weights, the **log posterior** looks like:

$$\log p(\theta | D) = - \sum_n \ell^{(\log)}(y_n, w \cdot x_n + b) - \sum_d \frac{1}{2\sigma^2} w_d^2 + \text{const} \quad \text{model definition} \quad (9.47)$$

$$= - \sum_n \ell^{(\log)}(y_n, w \cdot x_n + b) - \frac{1}{2\sigma^2} \|w\|^2 \quad (9.48)$$

and therefore reduces to a regularized logistic function, with a

squared 2-norm regularizer on the weights. (A 1-norm regularizer can be obtained by using a Laplace prior on  $w$  rather than a Gaussian prior on  $w$ .)

## 9.8 *Further Reading*

TODO